Ders 13 Genişletilmiş Notu: İkinci Ara Sınav Sorularının Tartışması: Sadakat Kartında State, Ders Kataloğunda Flyweight

Ders 13 Genişletilmiş Notu: İkinci Ara Sınav Sorularının Tartışması: Sadakat Kartında State, Ders Kataloğunda Flyweight

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

Bu kayıt dönemin son kaydıdır. Oturumun büyük kısmı bir sınava ayrıldığı için ders anlatımı yapılmamış, kaydın ilk yirmi dört dakikasında öğrencilerin soruları üzerine kod kusurlarının bulunması ve ikinci ara sınavın iki tasarım kalıbı sorusu tartışılmıştır. Not yalnız bu akademik tartışmayı aktarır.

İçindekiler

- 1. Kod Kusurlarını Kod Üzerinde Bulmak ve Adlandırmak
- 2. Soru 1: Havayolu Sadakat Kartı ve Kalıp Seçimi
- 3. State Kalıbı Neden En Uygun?
- 4. Chain of Responsibility Tartışması
- 5. Prototype Tartışması
- 6. Soru 2: Ders Kataloğu ve Flyweight
- 7. Diğer Kalıpların Soru 2'deki Sorunları
- 8. Flyweight, Kod Kalitesi ve Bakım Maliyeti

1. Kod Kusurlarını Kod Üzerinde Bulmak ve Adlandırmak

Hoca, bir öğrencinin sorusu üzerine kod kusuru ve refactoring konusunda bir kodun nasıl inceleneceğini anlattı (00:01:55). Elde bir ya da bir buçuk sayfalık bir kod olduğunu düşünelim. İnceleme şu adımlarla yapılır:

- Kod kusuru, kodun içinde geçtiği yer ya da yerlerde işaretlenir ve yanına bir numara konur.
- Ayrıca o numara için kusurun adı ve onu gidermek için uygulanacak refactoring eylemi yazılır. Örneğin: "1 numaralı yerde Long Method kod kusuru vardır; bunu düzeltmek için Extract Method eylemini uygulayacağım" ya da "Pull Up Method refactoring'ini uygulayacağım".
- En sonunda kodun düzgün tasarlanmış hâlinin UML sınıf şeması çizilir.

Bir kusurun kaç kez sayılacağı konusunda hoca iki örnek verdi. Bazı kod kusurları doğası gereği birden çok yere yayılır: Shotgun Surgery ve Divergent Change, birkaç yerde birden değişiklik yapma zorunluluğuyla ilgilidir. Kodun birkaç farklı yerindeki aynı kopya kod da tek bir kod kusurudur (Duplicated Code). Buna karşılık kodun üç ayrı yerinde üç ayrı metot uzunsa, bunlar üç ayrı Long Method kusurudur.

Hocanın sıraladığı değerlendirme ölçütleri aynı zamanda iyi bir kod incelemesinin ölçütleridir: doğru kusurun doğru yerde bulunması, kusurun adının orijinal (İngilizce) adıyla doğru yazıl-

ması, mantıklı bir refactoring eylemi önerilmesi, bu eylemin de orijinal adıyla yazılması ve son tasarımın doğru, mantıklı ve UML sınıf şeması kurallarına uygun olması.

> **Hocanın vurgusu:** Kod kusurlarının ve refactoring eylemlerinin adları orijinal adlarıyla, doğru yazılmalıdır.
>
> **Ders sonrası not:** Hoca Shotgun Surgery ile Divergent Change'i birlikte "birkaç yerde birden değişiklik yapma zorunluluğu" diye andı. Fowler'ın *Refactoring* kitabındaki tanımlar büyük olasılıkla şöyledir: Shotgun Surgery'de tek bir değişiklik birçok sınıfta küçük değişiklikler yapmayı gerektirir; Divergent Change'de ise tek bir sınıf farklı nedenlerle farklı biçimlerde değişmek zorunda kalır. Biri değişikliğin dağılması, öbürü bir sınıfta birden çok değişim nedeninin toplanmasıdır.

2. Soru 1: Havayolu Sadakat Kartı ve Kalıp Seçimi

İkinci ara sınavın ilk sorusu bir havayolu şirketinin sadakat kartı uygulamasıdır (soru metni ekrandan, 00:10:00; tam metin "Örnek Sorular" bölümünde). Yolcular Bronz düzeyde başlar, uçtukları mil başına puan kazanarak Gümüş ve Altın düzeyine geçer. Üst düzeylerde puan daha hızlı kazanılır: 100 mil başına Bronz'da 1, Gümüş'te 4, Altın'da 15 puan. Yıllık puan sınırına ulaşamayan yolcu bir alt düzeye düşer: 50 puan toplamayan Altın üye Gümüş'e, 25 puan toplayamayan Gümüş üye Bronz'a düşürülür; Bronz'dan aşağı düşülmez. Her düzeyin hangi hizmetlerden yararlandığı bir tabloyla verilir.

Hocanın çözüm notunda (ekrandan, 00:05:10) "+" tam uygun, "±" tam uygun değil demektir:

Kalıp	Değerlendirme	Not
State	+	
Strategy	+	İlk State düşünülür ama geçişler çok sık olmadığı için bu da olur.
Prototype	+	Derste yarım uygun sayılmasına karar verildi (bkz. Bölüm 5).
Chain of Responsibility	±	
Builder	±	
Abstract Factory	±	Karmaşıklığı gösteren UML şeması çözümün diğer sayfasında.

Hoca, Builder'ın da tam uymadığını; Builder ve Abstract Factory ile de bir şeyler çıkarılabileceğini ama sonuçta bunların yarım kaldığını söyledi (00:20:44).

3. State Kalıbı Neden En Uygun?

Hocanın açıklamasına göre State ve Strategy'de, özellikle de State'te, varlık kendi durum geçişlerini kendisi yapar (00:11:22). Yolcunun puanını, durum bilgisini ya da üyelik kurallarını etkileyebilecek bir getter, setter veya başka bir metot çağrıldığında geçiş denetimi nesnenin içinde yapılır ve nesne kendini bir durumdan ötekine rahatça geçirir.

Böyle yapılmazsa üyelik yükseltme ve düşürme işi dışarıdan yürütülmek zorunda kalır: veri tabanında belirli periyotlarla çalışan bir görev ya da ayrı bir sorgulama süreci tanımlanır; bu süreç bütün kayıtlara tek tek bakar ve yükselmesi ya da düşmesi gereken kişileri bulur. Bu zamanlanmış bir görev olsa da yine harici bir kaynaktan yapılması gereken bir iştir. State bu dış mekanizmayı gereksiz kılar.

> **Ek bilgi (derste anlatılmadı):** State ile Strategy'nin sınıf şemaları neredeyse aynıdır; fark niyettedir. State'te bağlam nesnesinin durumu değiştikçe davranışı değişir ve geçişleri durum nesneleri ya da bağlam kendisi yapar. Strategy'de ise algoritma genellikle dışarıdan seçilip verilir ve sık değişmez. Çözüm notundaki "geçişler çok sık olmadığı için Strategy de olur" ifadesi bu ayrıma dayanır.

4. Chain of Responsibility Tartışması

Bir öğrenci soruyu Chain of Responsibility ile çözdüğünü, sorunun derste görülen bir e-ticaret sitesi kampanya örneğine çok benzediğini söyledi (00:09:09). Başka bir öğrenci, üyeliklerin uçuş yapılmayınca düşüp yapılınca hiyerarşik olarak arttığını, bunun için zincire ayrı bir şey eklemek gerekebileceğini belirtti; ama bunun zinciri bozmadığını, uçuş yapılmadıysa azaltıp yapıldıysa artırarak düzeyin hesaplanabileceğini savundu. Bir ATM mantığı gibi düşünüldüğünde, örneğin 40 puanın kaç uçuştan geldiği önemsizdir; önemli olan bir eşiğe ulaşılıp ulaşılmadığı ve ulaşıldığında yolcunun karşılaşacağı kampanyalardır.

Hocanın yanıtı (00:11:22, 00:12:18): Chain of Responsibility'de istek normalde zincirde bir üst seviyeye iletilir. Ağacın alt seviyesine, yani zincirin öbür yönüne de bağlantı kurulursa düzey düşürme de belki kodlanabilir. Yine de konunun mantığına ve anlatış tarzına en uygun kalıp State'tir. Hoca, daha ince bir ayrım yapılsaydı Chain of Responsibility'nin "dörtte üç uygun" sayılabileceğini, ama yüzde yüz uygun olmayan her kalıbı bir alt kademede değerlendirdiğini söyledi.

Soru: Tek bir üyelik alanının Chain of Responsibility ile sağlanamaması bu çözümü yanlış kılar mı? **Cevap:** Yanlış kılmaz, ama yüzde yüz uygun da değildir; bu yüzden "tam uygun değil" kademesindedir (00:18:26).

5. Prototype Tartışması

5.1 Prototype bu soruda nasıl uygulanır?

Soru: Prototype bir nesnenin kalıbından farklı nesneler türetmek için kullanılır. Bu soruda neyi üretmek için kullanılacak? **Cevap (00:14:43):** Yolcu nesnesinde üyelik türü nesnesi bir üye alan olarak tutulur ve buna bir prototip atanır: bronz prototipi, gümüş prototipi ya da altın prototipi. Singleton burada olmaz, çünkü iki farklı bronz üyenin farklı durum bilgileri (ör. puanları) olacaktır; Singleton bunun yanlış kullanımı olur. Prototype ise uygun olabilir.

5.2 Prototype'ın dezavantajları

Bir öğrenci Prototype'a itiraz etti (00:15:27): Prototype değişmeyecek şeyler için kullanılır. Örneğin bir oyunda farklı bir canavar türü üretmek için o türün prototipi kopyalanır ve üretilen canavarın tür özellikleri sabit kalır. Oysa burada üyelik düzeyi (bronz, gümüş, altın) değişir. Prototype yalnız üyelik alanını temsil etmek ve atamak için kullanılabilir; uçuş yapılınca düzeyi düşürmek, bir şey yapılınca bir yere geçmek, düzeye göre kampanya göstermek gibi işler için çok daha fazla ek mantık gerekir. Öğrenci, Prototype'ın durum uzayı sınırlıysa kullanıldığını, durum uzayı çok küçükse Prototype'a girmenin başka kaynaklarda pek mantıklı görülmediğini de ekledi. Hoca bu dezavantajın doğru söylendiğini kabul etti.

Hoca bir dezavantaj daha ekledi (00:16:54): soruda altı ayrı hizmet kalemi var (bülten aboneliği, araba kiralama indirimi, öncelikli check-in vb.). Prototype'ta bu altı durum için altı `boolean` üye alan gerekir. Bronz prototipte bu alanların 2'si `true`, 4'ü `false`; Gümüş prototipte 4'ü `true`, 2'si `false` olur (tablodan sayım: Bronz'da 2 hizmet var, Gümüş'te 4, Altın'da 6). Bu yapılabilir ama çok uygun değildir. Bu tartışmanın sonunda hoca Prototype'ı yarım uygun saydığını söyledi.

5.3 Prototype'ta kalıtım yoktur

Bir öğrencinin çözümündeki sorunu anlatırken hoca önemli bir genel noktaya değindi (00:19:15): Prototype, kalıtım şeklinde çizilmiş; uçuş kartından Bronz kart, Gümüş kart, Altın kart alt sınıfları türetilmiş. Oysa Prototype kalıtım gerektirmeyen bir kalıptır. Üye alanların durumları değiştirilerek değişik prototip örnekleri oluşturulur ve bunlar bir kayıt alanına (registry), bir havuza kaydedilir. Alt sınıflarla çizilen ve havuzu göstermeyen bir şemada Prototype kalıbı görülemez.

> **Hocanın vurgusu:** Kalıtım yalnız gerekiyorsa kullanılmalıdır; gerekmediği yerde kullanılmaz.
>
> **Ek bilgi (derste anlatılmadı):** Gamma ve arkadaşlarının kitabındaki Prototype yapısında büyük olasılıkla `clone()` işlemini bildiren bir Prototype arayüzü ve onu

gerçekleştiren somut sınıflar yer alır. Hocanın vurguladığı nokta, her varyant (bronz, gümüş, altın) için ayrı bir alt sınıf açmak yerine aynı sınıfın farklı durumlu örneklerini bir prototip yöneticisinde (registry) tutup kopyalamaktır; kalıbın amaçlarından biri de alt sınıf sayısını azaltmaktır.

6. Soru 2: Ders Kataloğu ve Flyweight

İkinci soru, bir üniversitenin ders kataloğunun yönetildiği bir bilgi sistemidir: sistemde çok sayıda ders yer alacaktır, ancak bu derslerin ortak olan birçok özelliği bulunmaktadır (ekrandan, 00:10:00). Soruda bellek tasarrufunun önemli olduğu vurgulanmıştır.

Hocanın çözüm notu (ekrandan, 00:22:30): “Soru 2: Şu kalıplardan biri seçilebilir: + Flyweight. Bellek tasarrufunun önemli olduğu vurgulandığı için tek doğru çözüm Flyweight.”

Soru: Bellek tasarrufu vurgulandığı için tek doğru çözüm Flyweight denmiş; ama Prototype da bellek tasarrufu sağlıyor. İkisi arasında bu kadar büyük bir fark var mı? **Cevap (00:04:58):** Flyweight’in özü içsel (intrinsic) ve dışsal (extrinsic) bilgi ayrımıdır. İçsel bilgi havuzda (pool) paylaşılır, yalnız bir kez bulunur. Bir milyon nesne varsa bu bir milyon nesnenin yalnız dışsal bilgisi çoğul olur; içsel bilgisi tektir. Bunun alternatifi yoktur.

Soru: Bellek tasarrufu vurgulanmasaydı Prototype kullanılamaz mıydı? Bir sürü ders türü var ve bir dersten birden çok ders üretilebiliyor. **Cevap (00:05:37):** O zaman kullanılabilirdi. Bu yüzden Prototype diyen yanıt yarı doğru sayıldı.

> **Ek bilgi (derste anlatılmadı):** Prototype’ta her kopya kendi bütün durum bilgisini taşır; kopyalama nesne oluşturmayı kolaylaştırır ama bellekte nesne başına yer kaplamayı azaltmaz. Flyweight’te ise ortak (içsel) kısım yalnız bir kez tutulup bütün nesnelerce paylaşıldığı için bellek kazancı doğrudan bu ayrımdan gelir.

7. Diğer Kalıpların Soru 2’deki Sorunları

Çözüm notu diğer kalıpların sorunlarını tek tek sıralar (ekrandan, 00:21:20 ve 00:22:30).

7.1 Abstract Factory: kalıtım gruplaması ve kırılım sorunu

“Abstract Factory: Çok önemli kalıtım gruplaması yani kırılım sorunları var.” Notta bunu gösteren bir UML sınıf şeması bulunur (ekrandan, 00:21:20). Şemanın metne dökülmüş hâli:

```
«Abstract» DersFactory
  ├── ServisDersiFactory    [not: "Bunun zorunlu veya seçimlik olma durumu?"]
  ├── SeçimlikDersFactory
  └── ZorunluDersFactory
        - - -> ZorunluDersProjesi   (oluşturur)
        - - -> ZorunluDersSınavı    (oluşturur)

«Abstract» Proje
  ├── ZorunluDersProjesi
  └── SeçimlikDersProjesi

«Abstract» Sınav
  ├── ZorunluDersSınavı
  └── SeçimlikDersSınavı

[not, ürün hiyerarşilerine bağlı]:
  "Bu kategorileme,
   Ders dönemi: Güz, bahar
   Ders türü: Zorunlu, seçimlik
   vb'ye göre daha doğru kalıp mantığına göre,
   ama uygulama alanına göre yanlış bir modelleme."
```

Şemadaki sorun şudur: fabrikalar ders türüne göre (servis, seçimlik, zorunlu) ayrılmış, ürünler (proje, sınav) zorunlu ve seçimlik diye kırılmıştır. Servis dersinin kendisi zorunlu da seçimlik

de olabileceği için `ServisDersiFactory`'nin hangi ürün ailesini üreteceği belirsizdir. Ders dönemi, ders türü gibi birden çok eksene göre kategorileme kalıp mantığına göre doğru görünse de uygulama alanına göre yanlış bir modellemedir. Hoca bunu sözlü olarak şöyle özetledi (00:08:00): Abstract Factory'de kalıtım ilişkisini kurarken bir kategorileme yapılır; bu iş mantığına ve kalıp mantığına göre doğru olabilir ama uygulama alanına göre yanlış ya da eksik kalır.

7.2 Builder

"Builder: Problem alanına uygun ama bellek tasarrufu sağlamaz. Singleton ile desteklenirse bellek tasarrufu sağlayabilir ama bireysel ders nesnelerinin çeşitliliği desteklenmez." (ekrandan, 00:21:20)

7.3 Prototype

"Prototype: Üstteki ile aynı durum." (ekrandan, 00:21:20) Yani Prototype da problem alanına uygundur ama bellek tasarrufu sağlamaz.

8. Flyweight, Kod Kalitesi ve Bakım Maliyeti

Soru: Flyweight'i nasıl uygulayacağız? Zorunlu dersler, servis dersleri var; her birinin farklı özellikleri var. Hepsini Flyweight'te tek seferde mi uygulamalıyız? Ders zorunlu mu, ödevi var mı, projesi var mı gibi denetimlerle bu biraz if-else durumuna giriyor gibi. **Cevap (00:21:54):** Kusurlu (kokan) bir kod olabilir, doğru. Ama bu problemde bellek tasarrufu en önemli şey olduğu için, "doğru" modelleme yapılırsa işletim ve donanım masrafları çok daha fazla olacaktır. Bu yüzden kısa vadede parayı öncelemek için böyle bir karar verilir.

Hoca bakım maliyetinin de düşünülmesi gerektiğini ekledi. Hatalar ya da geliştirme istekleri geldiğinde, daha iyi bir tasarımla bakım bir birime mal oluyorken bu tasarımla bir buçuk birime mal oluyorsa, uzun vadede durum değişebilir. Yazılımın bir sonraki sürümünde Flyweight bırakılıp donanım masrafı göze alınabilir ve bu maliyet müşterilere satılan yazılımın fiyatına yansıtılabilir. Yani bir tasarım kalıbı kararı yalnız teknik değil, kısa ve uzun vadeli maliyetleri tartan bir karardır.

Örnek Sorular

Soru 1: Havayolu sadakat kartı (ekrandan, 00:10:00; çözüm notu ekrandan, 00:05:10)

Soru metni: Hezarfen Havayolları yeni kurulan bir havayolu taşımacılığı şirketi olarak müşterilerine bir sadakat kartı uygulaması başlatmak istemektedir. Bu uygulamaya kayıt olan yolcular önce Bronz düzeyde başlayarak uçtukları mil başına puan alarak daha üst düzeyler olan Gümüş ve Altın düzeyine geçebilir. Düzeylerde ilerledikçe daha hızlı puan kazanılacaktır. Örneğin Bronz düzeyde 100 mil başına 1 puan, Gümüş düzeyde 100 mil başına 4 puan, Altın düzeyde 100 mil başına 15 puan kazanılacaktır. Ancak her sene belli bir puan limitine ulaşamayan yolcunun üyelik düzeyi aşağı da düşebilecektir. Başlangıç düzeyi olan Bronz düzeyden daha aşağı düşülmeyecektir. Örneğin her yıl 50 puan toplamayan Altın üye Gümüş, 25 puan toplayamayan Gümüş üye ise Bronz düzeyine düşürülecektir. Üst düzey üyeliklerin alt düzey üyeliklere veya hiç üye olmamaya göre avantajları, verilen çeşitli hizmetlerden yararlanabilmektir:

	Bronz üyelik	Gümüş Üyelik	Altın Üyelik
Hezarfen bültenine ücretsiz abonelik	Var	Var	Var
Araba kiralama indirimi	Var	Var	Var
Öncelikli check-in	Yok	Var	Var
Özel bekleme salonundan faydalanma	Yok	Var	Var
Öncelikli bagaj alımı	Yok	Yok	Var
Uçuşta ücretsiz ikram	Yok	Yok	Var

Çözüm (derste verildi): Uygun kalıplar State ve Strategy'dir; Prototype sonradan yarım uygun sayıldı. Chain of Responsibility, Builder ve Abstract Factory tam uygun değildir; Singleton yanlıştır. State'te yolcu kendi üyelik geçişini kendisi yapar; puanı etkileyen her çağrıda geçiş denetimi nesnenin içinde yapılır ve dışarıdan periyodik bir tarama sürecine gerek kalmaz. Gerekçelerin ayrıntısı için Bölüm 2-5'e bakın.

Soru 2: Ders kataloğu bilgi sistemi (ekrandan, 00:10:00; çözüm notu ekrandan, 00:21:20 ve 00:22:30)

Soru metni: Üniversitenizin ders katalogunun yönetildiği bir bilgi sistemi hazırlıyorsunuz. Sistemde çok sayıda ders yer alacaktır, ancak bu derslerin ortak olan birçok özelliği bulunmaktadır. Örneğin zorunlu derslerin …

Çözüm (derste verildi): Bellek tasarrufunun önemli olduğu vurgulandığı için tek doğru çözüm Flyweight'tir. Derslerin ortak özellikleri içsel bilgi olarak havuzda bir kez tutulur ve paylaşılır; derse özgü bilgiler dışsal bilgidir. Abstract Factory'de kalıtım gruplaması (kırılım) sorunu vardır; Builder ve Prototype problem alanına uygun ama bellek tasarrufu sağlamaz.

Soru 3: Prototype ile Flyweight (derste sözlü, 00:04:30)

Soru metni: Prototype de bellek tasarrufu sağlıyor. Bellek tasarrufu açısından Prototype ile Flyweight arasında büyük bir fark var mı?

Çözüm (derste verildi): Flyweight durum bilgisini içsel ve dışsal diye ayırır; içsel bilgi havuzda tek kopya olarak paylaşılır, bir milyon nesne olsa bile yalnız dışsal bilgi bir milyon kez tutulur. Prototype bu ayrımı yapmaz. Bellek tasarrufu vurgulanmasaydı Prototype kullanılabilirdi.

Soru 4: Sadakat kartında Singleton (derste sözlü, 00:14:43)

Soru metni: Sadakat kartı sorusunda üyelik düzeyleri için Singleton kullanılabilir mi?

Çözüm (derste verildi): Kullanılmaz. İki farklı bronz üyenin farklı durum bilgileri (ör. puanları) olacaktır; tek bir paylaşılan nesne bunu taşıyamaz. Üyelik alanına bronz, gümüş ya da altın prototipinin bir kopyası atanabilir.

Dersin Toparlaması

Bu son kayıtta iki temel ders öne çıktı. Birincisi, kalıp seçimi problemin özüne bakılarak yapılır: sadakat kartında özü zamanla değişen bir durum ve ona bağlı davranış olduğu için State en uygun kalıptır; ders kataloğunda özü çok sayıdaki benzer nesnenin bellek maliyeti olduğu için Flyweight'in alternatifi yoktur. Chain of Responsibility, Prototype, Builder ve Abstract Factory gibi kalıplar bu problemlere kısmen uyar ama ya yön (zincirin tek yönlü olması), ya veri yapısı (boolean alanlarla şişen prototipler), ya da kategorileme (uygulama alanına uymayan kırılım) açısından eksik kalır. İkincisi, kalıtım yalnız gerektiğinde kullanılmalıdır; Prototype'ın özü alt sınıflar değil, farklı durumlu örneklerin bir havuzda tutulup kopyalanmasıdır.

Hoca ayrıca bir tasarım kararının maliyet boyutunu hatırlattı: Flyweight gibi bellek odaklı bir çözüm kodu kusurlu hâle getirebilir; kısa vadeli donanım tasarrufu ile uzun vadeli bakım maliyeti tartılarak karar verilir. Kod kusurları konusunda da her kusurun doğru yerde, doğru sayıda ve orijinal adıyla belirtilmesi, karşısına da orijinal adıyla uygun refactoring eyleminin yazılması gerektiğini vurguladı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.